Google Playstore EDA

1.0 About Author

- Project: Google Play Store Apps
- Author: Muhammad Waqas
- Code Submision Date: 17-02-2024
- Author's Contact Info:
  - **Email:** waqasliaqat630@gmailcom
  - Linkedin
  - Github
  - kaggle

2.0 About Data

2.1 MetaData

- Data: Google Play Store Apps
- Data Age: The data was collected in the month of june 2021.
- Dataset: link

Columns Information

Column Name	Description
App Name	Name of the app
App Id	Package name
Category	App category
Rating	Average rating
Rating Count	Number of rating
Installs	Approximate install count
Minimum Installs	Approximate minimum app install count
Maximum Installs	Approximate maximum app install count
Free	Whether app is Free or Paid
Price	App price
Currency	App currency
Size	Size of application package
Minimum Android	Minimum android version supported
Developer Id	Developer Id in Google Playstore
Developer Website	Website of the developer
Developer Email	Email-id of developer
Released	App launch date on Google Playstore
Last Updated	Last app update date
Content Rating	Maturity level of app
Privacy Policy	Privacy policy from developer
Ad Supported	Ad support in app
In App Purchases	In-App purchases in app
Editors Choice	Whether rated as Editor Choice

Scraped Time	Scraped date-time in GMT

2.2 Task

We intend to conduct an Exploratory Data Analysis (EDA) on the given dataset. The EDA will serve as the basis for the necessary Data Wrangling activities to be carried out for the purposes of data cleaning and normalization. During the coding process, we will document our observations. Ultimately, we will produce a summary and draw conclusions from our findings.

2.3 Objective

- The primary aim of this project is to conduct a thorough analysis of the dataset to identify significant insights.
- The ultimate goal is to project and emphasize customer dynamics and demands to developers and other relevant stakeholders.
- This will assist in driving more business towards their upcoming applications.

2.4 Kernel Version Used:

- Python 3.12.0

3.0 import libraries

- We will use following libraries:
  1. Pandas: For manipulation and analysis of data.
  2. Numpy: For array manipulation & math functions for summarizing, transforming, visualizing data (stats, distributions, correlations).
  3. Seaborn: For beautiful, informative statistical plots
  4. Matplotlib: For informative & customizable visualizations

In [1]:
```
import pandas as pd
import numpy as np
# visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
# Enable inline plotting for convenience within the Jupyter notebook
%matplotlib inline
# use case of warnings.filterwarnings('ignore') is to suppress warnings in order to prevent them from being dis
import warnings
warnings.filterwarnings('ignore')
```

4.0 Data Loading, Exploration and Wrangling

we will proceed as follows:

4.1 Load Dataset

In [2]:
```
df=pd.read_csv("data.csv")
```

4.2 Increase Readability of Wide DataFrames

When working with large datasets, DataFrames can easily contain more columns and rows than can comfortably fit on your screen. This can make it difficult to read and understand the data. So we will use the following code snippet, so that no valuable information can be overlooked.

In [3]:
```
pd.set_option('display.max_columns',None)
pd.set_option('display.max_rows',None)
```

4.3 Get sneak Pek of data

- With this piece of code we can get some sense of data, by viewing top rows.
- Also it saves our time of getting into large file of data to get sense of data.

In [4]:
```
df.head()
```

Out[4]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Currency	Size
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0	15	True	0.0	USD	10M
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	USD	2.9M
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	USD	3.7M
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	USD	1.8M
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0	478	True	0.0	USD	6.2M

4.4 View column names

In [5]:
```
df.columns
```

Out[5]:
```
Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs',
       'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency',
       'Size', 'Minimum Android', 'Developer Id', 'Developer Website',
       'Developer Email', 'Released', 'Last Updated', 'Content Rating',
       'Privacy Policy', 'Ad Supported', 'In App Purchases', 'Editors Choice',
       'Scraped Time'],
      dtype='object')
```

4.5 View Shape of dataset

In [6]:
```
print(f"The dataframe has {df.shape[0]} rows and {df.shape[1]} columns")
```

```
The dataframe has 2312944 rows and 24 columns
```

4.6 Data information

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2312944 entries, 0 to 2312943
Data columns (total 24 columns):
 #   Column             Dtype
---  ------             -----
 0   App Name           object
 1   App Id             object
 2   Category           object
 3   Rating             float64
 4   Rating Count       float64
 5   Installs           object
 6   Minimum Installs   float64
 7   Maximum Installs   int64
 8   Free               bool
 9   Price              float64
 10  Currency           object
 11  Size               object
 12  Minimum Android    object
 13  Developer Id       object
 14  Developer Website  object
 15  Developer Email    object
 16  Released           object
 17  Last Updated       object
 18  Content Rating     object
 19  Privacy Policy     object
 20  Ad Supported       bool
 21  In App Purchases   bool
 22  Editors Choice     bool
 23  Scraped Time       object
dtypes: bool(4), float64(4), int64(1), object(15)
memory usage: 361.8+ MB
```

Observation set 1:

1. We have 4 bool, 4 float, 1 int and 15 object columns in our dataset.
2. 'Released' and 'Last Updated' should be converted to datetime.
3. We should convert "Size" column to numerical with name size_in_mb.
4. "Scrapped Time" is totally irrelevent feature in our apps dataset, it should be removed.

4.7 Missing Values

4.7.1 Finding Missing Values

In [8]:
```
df.isnull().sum().sort_values(ascending=False)
```

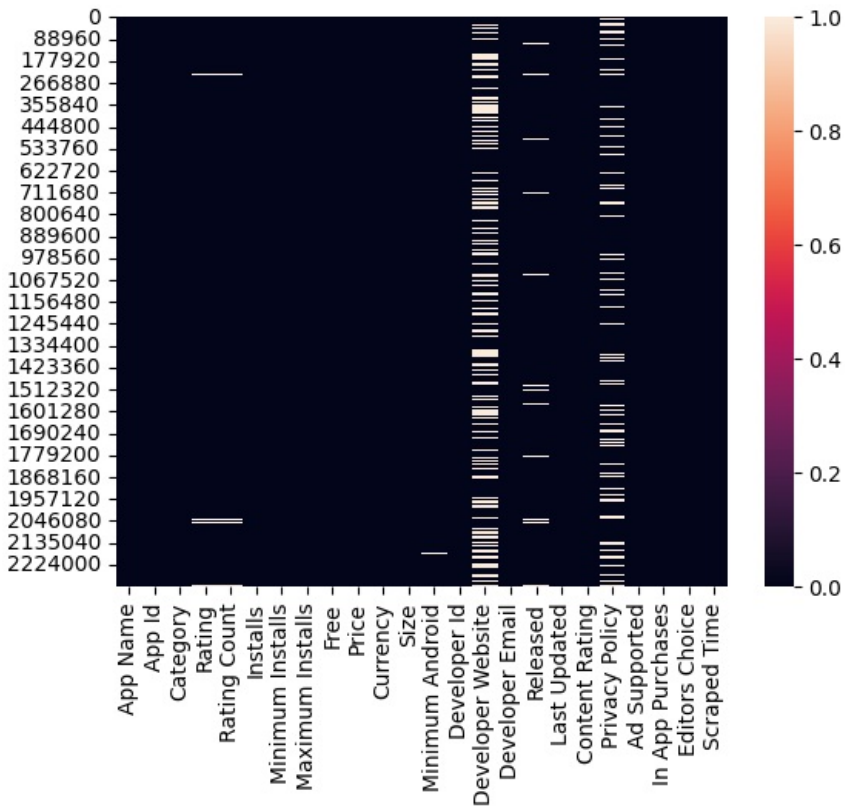
Out[8]:
```
Developer Website    760835
Privacy Policy       420953
Released              71053
Rating                22883
Rating Count          22883
Minimum Android        6530
Size                    196
Currency                135
Installs                107
Minimum Installs        107
Developer Id             33
Developer Email          31
App Name                  5
App Id                    0
Price                     0
Free                      0
Maximum Installs          0
Last Updated              0
Content Rating            0
Category                  0
Ad Supported              0
In App Purchases          0
Editors Choice            0
Scraped Time              0
dtype: int64
```

In [9]:
```
((df.isnull().sum()/len(df))*100).sort_values(ascending=False)
```

Out[9]:
```
Developer Website    32.894657
Privacy Policy       18.199879
Released              3.071972
Rating                0.989345
Rating Count          0.989345
Minimum Android       0.282324
Size                  0.008474
Currency              0.005837
Installs              0.004626
Minimum Installs      0.004626
Developer Id          0.001427
Developer Email       0.001340
App Name              0.000216
App Id                0.000000
Price                 0.000000
Free                  0.000000
Maximum Installs      0.000000
Last Updated          0.000000
Content Rating        0.000000
Category              0.000000
Ad Supported          0.000000
In App Purchases      0.000000
Editors Choice        0.000000
Scraped Time          0.000000
dtype: float64
```

In [10]:
```
sns.heatmap(df.isnull())
```

Out[10]:
```
<Axes: >
```



4.7.2 Handling Missing Values

In [11]:
```
# quick overview of data
df.head()
```

Out[11]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Currency	Size
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0	15	True	0.0	USD	10M
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	USD	2.9M
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	USD	3.7M
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	USD	1.8M
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0	478	True	0.0	USD	6.2M

- As Developer Website, Privacy Policy, Developer Id and Developer Email are null, most probably because developer may not have these. We can fill them with "not present".

In [12]:
```
# filing null with "not present"
df["Developer Website"]=df["Developer Website"].fillna(value="not present")
df["Privacy Policy"]=df["Privacy Policy"].fillna(value="not present")
df["Developer Id"]=df["Developer Id"].fillna(value="not present")
df["Developer Email"]=df["Developer Email"].fillna(value="not present")
# have a view of these refreshed rows after filling missing.
df[["Developer Website", "Privacy Policy", "Developer Id", "Developer Email"]].isnull().sum()
```

Out[12]:
```
Developer Website    0
Privacy Policy       0
Developer Id         0
Developer Email      0
dtype: int64
```

App Names & Minimum Androids

- As there are just some missing app names and minimum android, Hence we can drop that null rows

In [13]:
```
#Dropping null rows
df=df.dropna(subset=["App Name"])
df=df.dropna(subset=["Minimum Android"])
# let's check
df[["App Name","Minimum Android"]].isnull().sum()
```

Out[13]:
```
App Name           0
Minimum Android    0
dtype: int64
```

Size

In [14]:
```
df[df["Size"].isnull()].head()
```

Out[14]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Pric
37031	Change The World Together: My Travel Friends V...	com.mytravelfriends.changetheworldtogether	Education	NaN	NaN	1+	1.0	3	True	0
60487	Style Salon Premium App (for demo purpose)	com.gobony.premium.app	Beauty	NaN	NaN	5+	5.0	5	True	0
82844	Rolling Muse	com.rollingmuse	Music & Audio	NaN	NaN	50+	50.0	59	True	0
83346	ArabiConnect Token	com.arabbanksofttoken	Finance	NaN	NaN	5+	5.0	5	True	0
100136	Backyard Bash: New Match 3 Pet Game	com.bigfishgames.backyardbashgoog	Casual	NaN	NaN	100,000+	100000.0	133062	True	0

In [15]:
```
print(f"Total missing values in size are: {len(df[df['Size'].isnull()])}")
print(f"Total number of values having missing values and varies with devices both, is: {len(df[(df['Size'].isnu
```

```
Total missing values in size are: 196
Total number of values having missing values and varies with devices both, is: 196
Total number of values having missing values and varies with devices both, is: 196
```

In [16]:
```
df[df["Minimum Android"]=="Varies with device"].head()
```

Out[16]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Currency	Si
115	VM-Delay	com.irv.vm_delay	Tools	NaN	NaN	50+	50.0	71	True	0.0	USD	Vari w devi
180	Shop 13	com.shop1313	Shopping	4.3	18.0	1,000+	1000.0	2340	True	0.0	USD	Vari w devi
210	Promotor	com.touchesbegan.promotor	Productivity	NaN	NaN	100+	100.0	135	True	0.0	USD	Vari w devi
284	xrsta xatr	com.xrsta.xatr	Entertainment	NaN	NaN	10+	10.0	14	True	0.0	USD	Vari w devi
286	Goodfellas	com.sapanbhuta.goodfellas	Lifestyle	4.0	8.0	1,000+	1000.0	1628	True	0.0	USD	Vari w devi

- As we can see that **Size** have relation with **minimum android** and on all the null **size** points **minimum android** have "Varies with devices".
- In second snippet of code we can see that in case of "varies with device" in **minimum android**, "varies with devices" is also present in **size** .
- We can fill nulls of **Size** with "Varies with device".

In [17]:
```
df["Size"]=df["Size"].fillna(value="Varies with device")
```

Currency

In [18]:
```
df["Currency"].unique()
```

Out[18]:
```
array(['USD', 'XXX', 'CAD', 'EUR', 'INR', nan, 'VND', 'GBP', 'BRL', 'KRW',
       'TRY', 'RUB', 'SGD', 'AUD', 'PKR', 'ZAR'], dtype=object)
```

In [19]:
```
print(f"Total missing values in Currency are: {len(df[df['Currency'].isnull()])}")
print(f"Total number of datapoints having missing values and price 0.0 both, is: {len(df[(df['Currency'].isnull
```

```
Total missing values in Currency are: 135
Total number of datapoints having missing values and price 0.0 both, is: 135
```

- We can see that all the nulls in currency have 0 price, hence we can impute currency with XXX.

In [20]:
```
df["Currency"]=df["Currency"].fillna(value="XXX")
```

```
# now let's check:
df["Currency"].isnull().sum()
```

Out[20]: 0

Installs, Minimum Installs,rating and rating count

- As missing percentage is very small in them and each two rows have missing on similar rows, hence we can drop these null values by dropping from every second.

In [21]:
```
df=df.dropna(subset=["Installs"])
df=df.dropna(subset=["Rating"])
# let's check
df[["Installs","Rating"]].isnull().sum()
```

Out[21]:
```
Installs    0
Rating      0
dtype: int64
```

Released

- We can predict released from last updated date, but it can manipulate some data because famous apps are updated a lot.
- Hence we decided to drop the null values.

In [22]:
```
df=df.dropna(subset=["Released"])
# let's check
df["Released"].isnull().sum()
```

Out[22]: 0

In [23]:
```
df.isnull().sum().sort_values(ascending=False)
```

Out[23]:
```
App Name             0
App Id               0
Editors Choice       0
In App Purchases     0
Ad Supported         0
Privacy Policy       0
Content Rating       0
Last Updated         0
Released             0
Developer Email      0
Developer Website    0
Developer Id         0
Minimum Android      0
Size                 0
Currency             0
Price                0
Free                 0
Maximum Installs     0
Minimum Installs     0
Installs             0
Rating Count         0
Rating               0
Category             0
Scraped Time         0
dtype: int64
```

Milestone 1: We have cleaned the dataset from null values.

4.8 Duplicates

In [24]:
```
df.duplicated().any()
```

Out[24]: False

In [25]:
```
df["App Name"].duplicated().any()
```

Out[25]: True

In [67]:
```
df["App Name"].value_counts().sort_values(ascending=False).head()
```

Out[67]:
```
App Name
Tic Tac Toe       237
Flashlight        198
Calculator        164
Age Calculator    148
Sudoku            143
Name: count, dtype: int64
```

In [27]:
```
df[df["App Name"]=="Tic Tac Toe"].head()
```

Out[27]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Currency	Size	Mir Ar
3918	Tic Tac Toe	com.dev.atte.tic_tac_toe	Puzzle	0.0	0.0	10+	10.0	30	True	0.0	USD	3.4M	4
13758	Tic Tac Toe	com.md.tictactoe	Board	0.0	0.0	10+	10.0	31	True	0.0	USD	34M	4
25351	Tic Tac Toe	com.sthirumuru.tictactoenew	Board	4.2	11.0	5,000+	5000.0	6379	True	0.0	USD	7.4M	7
28142	Tic Tac Toe	com.fsk00x.TicTacToe	Board	0.0	0.0	500+	500.0	922	True	0.0	USD	8.6M	4
30962	Tic Tac Toe	net.snatchdreams.tictactoe	Puzzle	0.0	0.0	10+	10.0	38	True	0.0	USD	927k	4

Milestone 2: Hence no duplicates found

- Although App_name may have duplications, the records are unique as a whole.

4.9 Features Engineering

Size

- Converting "Size" into numeric format.

In [28]:
```
# unique values in size
df["Size"].unique()
```

Out[28]:
```
array(['10M', '2.9M', '3.7M', ..., '405M', '3.2k', '512M'], dtype=object)
```

In [29]:
```
# number of apps with size in KB.
having_k=df["Size"].str.contains("k").sum()
# number of apps with size in MB
having_M=df["Size"].str.contains("M").sum()
# number of apps with size varying wih device
having_varies=df["Size"].str.contains("Varies with device").sum()
print(having_k+having_M+having_varies)
print(len(df["Size"]))
```

```
2235314
2235326
```

- It means "Size" not only contains just 3 types of sizes but also contains another one, we need to find that out.

In [30]:
```
# find apps not consisting of KB, MB or varies with device
df[(~df["Size"].str.contains("k"))& (~df["Size"].str.contains("M"))& (~df["Size"].str.contains("Varies with dev
```

Out[30]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Currency	Size
93175	Titan Quest: Legendary Edition	com.hg.titanquestedition	Role Playing	4.1	1387.0	5,000+	5000.0	8329	False	19.99	USD	1.5G
293625	魔力少女-元氣蘿莉寶貝水藍手遊私服,這是召喚師奇幻題材,角色扮演卡牌掛機私服遊戲.	com.mofa.moli.meishaonv	Casual	4.9	90.0	1,000+	1000.0	1602	True	0.00	USD	1.1G
479954	Kỳ Nữ Hoàng Cung	vn.funtap.kynuhoangcung	Role Playing	3.8	6093.0	500,000+	500000.0	573342	True	0.00	USD	1.0G

- That fourth type of size is "G"(GB).

In [31]:
```
having_g=df["Size"].str.contains("G").sum()
print(having_k+having_M+having_varies+having_g)
print(len(df["Size"]))
```

```
2235326
2235326
```

- Now we will convert "Size" to numeric having size in MB.

In [32]:
```
# Apps with size less than one GB but greater than 1000 Mbs needed to be removed with commas
df["Size"]=df["Size"].str.replace(",","")
```

In [33]:
```
# creating a new column with name size in mb

#let's define a function
def convert_size(Size):
    if isinstance(Size, str):
        if 'k' in Size:
            return float(Size.replace('k',''))/1024
        elif 'M' in Size:
            return float(Size.replace('M',''))
        elif 'G' in Size:
            return float(Size.replace('G',''))*1024
        elif 'Varies with device' in Size:
            return np.nan
    return Size
df['size_in_MB']=df['Size'].apply(convert_size)
```

- Here we can see that all the size values are in MB now.
- None values represent app with unknown size, "varies with device" was also showing uknown size.

Released and Last Updated

Converting "Released" and "Last Updated" into datetime

In [34]:
```
df[["Released","Last Updated"]].head()
```

Out[34]:

	Released	Last Updated
0	Feb 26, 2020	Feb 26, 2020
1	May 21, 2020	May 06, 2021
2	Aug 9, 2019	Aug 19, 2019
3	Sep 10, 2018	Oct 13, 2018
4	Feb 21, 2020	Nov 12, 2018

In [35]:
```
# converting Both columns to datetime
df["Released"]=pd.to_datetime(df["Released"])
df["Last Updated"]=pd.to_datetime(df["Last Updated"])
```

Scrapped Time

- removing "Scraped Time"

In [36]:
```
df.drop(columns=["Scraped Time"],axis=1,inplace=True)
```

In [37]:
```
# Let's check the dataframe again
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2235326 entries, 0 to 2312943
Data columns (total 24 columns):
 #   Column             Dtype
---  ------             -----
 0   App Name           object
 1   App Id             object
 2   Category           object
 3   Rating             float64
 4   Rating Count       float64
 5   Installs           object
 6   Minimum Installs   float64
 7   Maximum Installs   int64
 8   Free               bool
 9   Price              float64
 10  Currency           object
 11  Size               object
 12  Minimum Android    object
 13  Developer Id       object
 14  Developer Website  object
 15  Developer Email    object
 16  Released           datetime64[ns]
 17  Last Updated       datetime64[ns]
 18  Content Rating     object
 19  Privacy Policy     object
 20  Ad Supported       bool
 21  In App Purchases   bool
 22  Editors Choice     bool
 23  size_in_MB         float64
dtypes: bool(4), datetime64[ns](2), float64(5), int64(1), object(12)
memory usage: 366.7+ MB
```

Milestone 3: Relavent data type assigned to all features.

5.0 Exploring all features

5.1 Descriptive summary

In [38]:
```
df.describe().T
```

Out[38]:

	count	mean	min	25%	50%	75%	max	std
Rating	2235326.0	2.206446	0.0	0.0	3.0	4.3	5.0	2.10836
Rating Count	2235326.0	2759.854709	0.0	0.0	6.0	42.0	138557570.0	198757.431682
Minimum Installs	2235326.0	176026.619731	0.0	50.0	500.0	5000.0	10000000000.0	14203877.2196
Maximum Installs	2235326.0	308113.528367	0.0	86.0	706.0	7427.0	12057627016.0	22088564.214633
Price	2235326.0	0.104767	0.0	0.0	0.0	0.0	400.0	2.660894
Released	2235326	2018-08-29 01:31:21.019591680	2010-01-28 00:00:00	2017-07-01 00:00:00	2019-03-19 00:00:00	2020-04-15 00:00:00	2021-06-16 00:00:00	NaN
Last Updated	2235326	2019-12-03 14:46:04.660903424	2009-02-09 00:00:00	2019-04-15 00:00:00	2020-05-14 00:00:00	2021-02-04 00:00:00	2021-06-16 00:00:00	NaN
size_in_MB	2184196.0	19.143454	0.003125	4.9	10.0	25.0	1536.0	23.915915

Observation set 2:

- Minimum size of app is 3KB and maximum size is 1536MB.
- AVERAGE size is 10MB.
- More than 75% apps are free.
- According to this dataset first app was released on 2010-01-28 and latest app(till this dataset collected) was released on 2021-06-16.
- Oldest updated app date is 2009-02-09 however the first app was released on 2010-01-28, it is impossible for an app being updated before it's release, It is anomaly and we will try to find more anomalies like this.

In [39]:
```
df.describe(include="all")
```

Out[39]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	
count	2235326	2235326	2235326	2.235326e+06	2.235326e+06	2235326	2.235326e+06	2.235326e+06	2235326	2.23
unique	2104582	2235326	48	NaN	NaN	22	NaN	NaN	2	
top	Tic Tac Toe	com.ishakwe.gakondo	Education	NaN	NaN	100+	NaN	NaN	True	
freq	381	1	235582	NaN	NaN	429307	NaN	NaN	2191389	
mean	NaN	NaN	NaN	2.206446e+00	2.759855e+03	NaN	1.760266e+05	3.081135e+05	NaN	1.04
min	NaN	NaN	NaN	0.000000e+00	0.000000e+00	NaN	0.000000e+00	0.000000e+00	NaN	0.00
25%	NaN	NaN	NaN	0.000000e+00	0.000000e+00	NaN	5.000000e+01	8.600000e+01	NaN	0.00
50%	NaN	NaN	NaN	3.000000e+00	6.000000e+00	NaN	5.000000e+02	7.060000e+02	NaN	0.00
75%	NaN	NaN	NaN	4.300000e+00	4.200000e+01	NaN	5.000000e+03	7.427000e+03	NaN	0.00
max	NaN	NaN	NaN	5.000000e+00	1.385576e+08	NaN	1.000000e+10	1.205763e+10	NaN	4.00
std	NaN	NaN	NaN	2.108360e+00	1.987574e+05	NaN	1.420388e+07	2.208856e+07	NaN	2.66

Observation set 3:

- Out of 2235326 apps, 2104582 app names are unique. "Tic Tac Toe", is the most copied names.
- There are 48 categories of apps and education category have most number of apps.
- most of the apps have downloads in hundreds.
- There are 13 type of currencies, and "USD" is the most common.
- "4.1 and up" is the most common android version.
- most of the developers don't have their own website.
- Most of the apps are ad supported.
- Most of the apps don't allow in app purchase.

5.2 Released and last_Updated dates Anomalies

In [40]:
```
(len(df[df["Released"]>df["Last Updated"]])/len(df))*100
```

Out[40]:
```
0.4396226769607654
```

- As their percentage is very small, so I deided to keep only those entries where released date is less than last updated date.

In [41]:
```
df=df[df["Released"]<df["Last Updated"]]
```

App age column

In [42]:
```
today=pd.to_datetime("2021-06-16")
df["age_in_month"]=(today-df["Released"]).dt.days // 30
df.head()
```

Out[42]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Cu
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	
5	IMOCCI	com.imocci	Social	0.0	0.0	50+	50.0	89	True	0.0	
6	unlimited 4G data prank free app	getfreedata.superfatiza.unlimitedjiodataprank	Libraries & Demo	4.5	12.0	1,000+	1000.0	2567	True	0.0	

6.0 Question and Aswers :

We are going to pose following questions against the dataset:

- **Q1.** Is there any correlations between rating and dowloads?
- **Q2:** Is there any relationship between downloads and app type(free/paid)?
- **Q3.** Which TOP 5 app categories have the most number of apps?
- **Q4.** Which TOP 5 app categories have the most downloads?
- **Q5.** How does the price of an app (free vs. paid) affect its rating?
- **Q6.** How does app size relate to installs?
- **Q7.** Are Size and Rating dependent on each other?
- **Q8.** Does the content rating of an app (e.g., Everyone, Teen, Mature) influence its install base?
- **Q9.** Which Year has the most number of apps released?
- **Q10.** Apps released in Which Year could get the most number of app downloads?
- **Q11:** Does app age influences it's downoads?
- **Q12.** Show interdependency of all numerical features.

In [43]:
```
df.head()
```

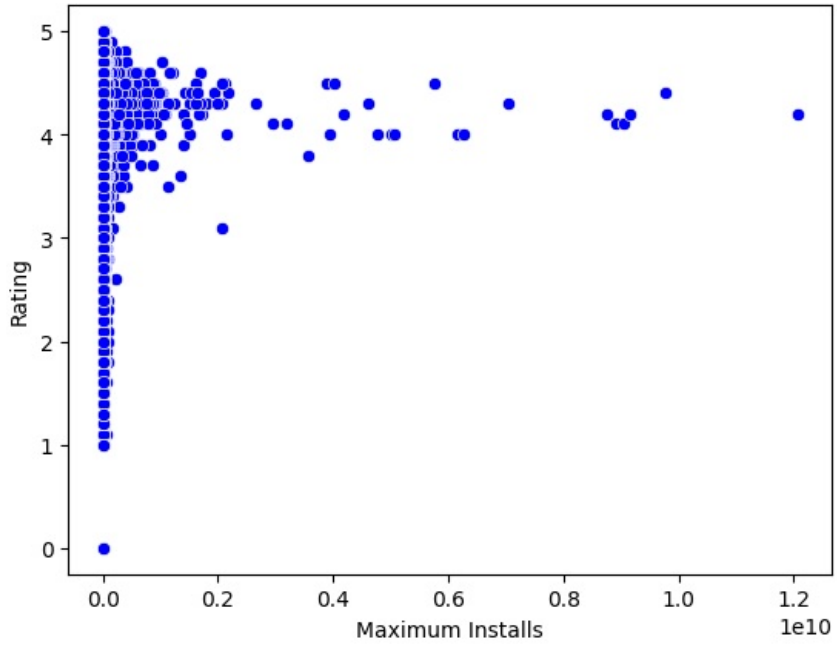
Out[43]:

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	Cu
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	
5	IMOCCI	com.imocci	Social	0.0	0.0	50+	50.0	89	True	0.0	
6	unlimited 4G data prank free app	getfreedata.superfatiza.unlimitedjiodataprank	Libraries & Demo	4.5	12.0	1,000+	1000.0	2567	True	0.0	

Q1. Is there any correlations between rating and dowloads?

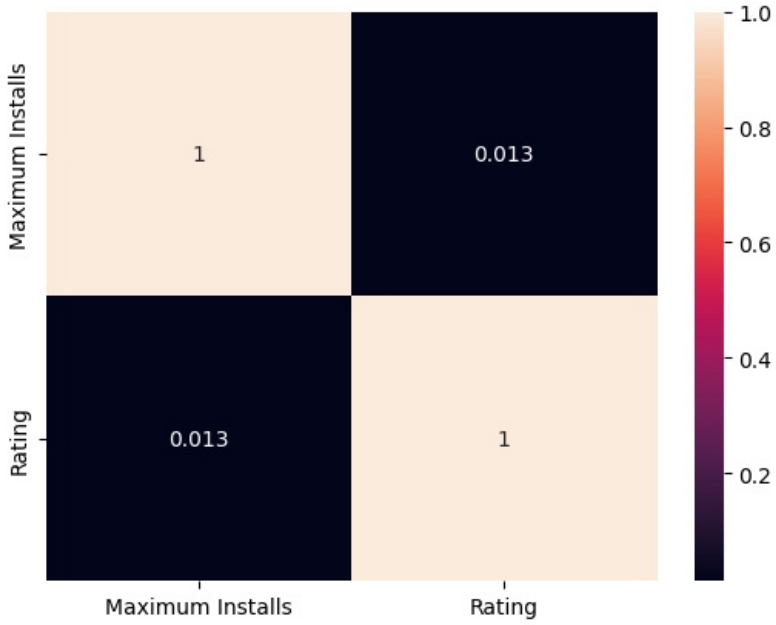
In [44]:
```
sns.scatterplot(data=df,x="Maximum Installs",y="Rating",c="blue")
```

Out[44]:
```
<Axes: xlabel='Maximum Installs', ylabel='Rating'>
```



In [45]:
```
sns.heatmap(df[["Maximum Installs","Rating"]].corr(),annot=True)
```

Out[45]: <Axes: >



Ans 1: Not seen any relationship in apps with less downloads, but it is seen that apps with more downloads have quit decent rating.

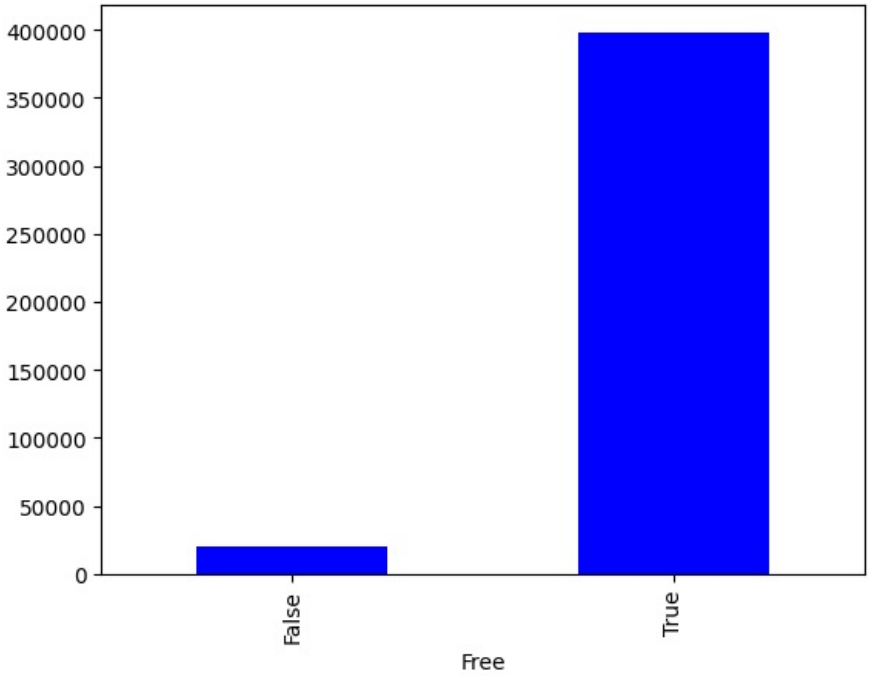
Q2: Is there any relationship between downloads and app type(free/paid)?

In [46]:
```
df.groupby("Free")["Maximum Installs"].mean()
```

Out[46]:
```
Free
False     19730.857384
True     398234.160508
Name: Maximum Installs, dtype: float64
```

In [47]:
```
df.groupby("Free")["Maximum Installs"].mean().plot(kind="bar",color="blue")
```

Out[47]: <Axes: xlabel='Free'>



Ans 2: Free apps generally have more downloads than that of paid ones

Q3. Which TOP 5 app categories have the most number of apps?

In [48]:
```
df["Category"].unique()
```

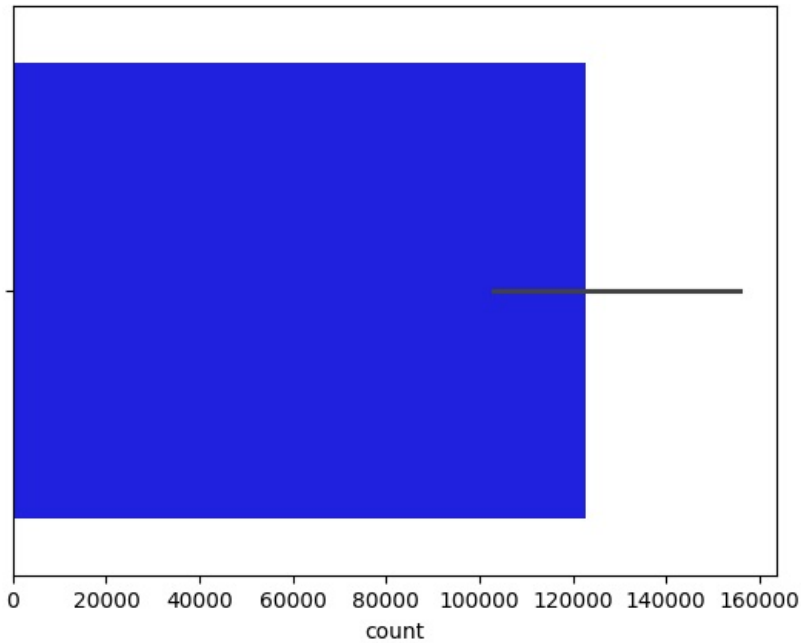
Out[48]:
```
array(['Tools', 'Productivity', 'Communication', 'Social',
       'Libraries & Demo', 'Personalization', 'Maps & Navigation',
       'Travel & Local', 'Food & Drink', 'Lifestyle', 'Books & Reference',
       'Medical', 'Puzzle', 'Entertainment', 'Arcade', 'Auto & Vehicles',
       'Photography', 'Health & Fitness', 'Education', 'Shopping',
       'Board', 'Music & Audio', 'Sports', 'Beauty', 'Business',
       'Finance', 'News & Magazines', 'Casual', 'House & Home', 'Racing',
       'Card', 'Events', 'Trivia', 'Weather', 'Adventure', 'Strategy',
       'Word', 'Action', 'Music', 'Video Players & Editors', 'Simulation',
       'Dating', 'Role Playing', 'Art & Design', 'Educational', 'Casino',
       'Comics', 'Parenting'], dtype=object)
```

In [49]:
```
top_numbers=df.Category.value_counts().sort_values(ascending=False).head()
top_numbers
```

Out[49]:
```
Category
Education        186858
Tools            109972
Business         109721
Music & Audio    108859
Entertainment     98586
Name: count, dtype: int64
```

In [50]:
```
sns.barplot(x=top_numbers,color="blue")
```

Out[50]:
```
<Axes: xlabel='count'>
```



Ans 3: Education, Music & Audio, Tools, Business and Entertainment have the most number of apps.

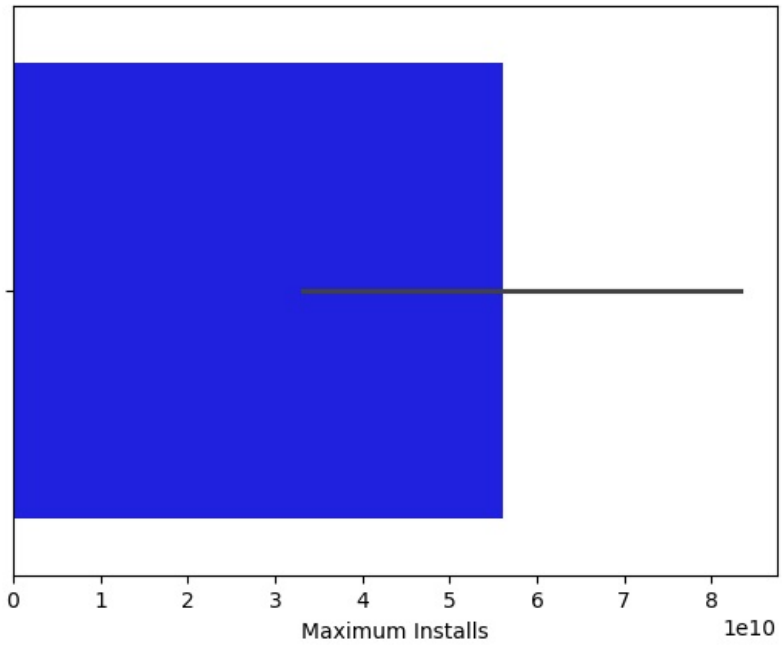
Q4. Which TOP 5 app categories have the most downloads?

In [51]:
```
top_downloads=df.groupby("Category")["Maximum Installs"].sum().sort_values(ascending=False).head()
top_downloads
```

Out[51]:
```
Category
Tools                      105033880160
Communication               70832882224
Productivity                43814986522
Photography                 30659045949
Video Players & Editors     30408149074
Name: Maximum Installs, dtype: int64
```

In [52]:
```
sns.barplot(x=top_downloads,color="blue")
```

Out[52]:
```
<Axes: xlabel='Maximum Installs'>
```

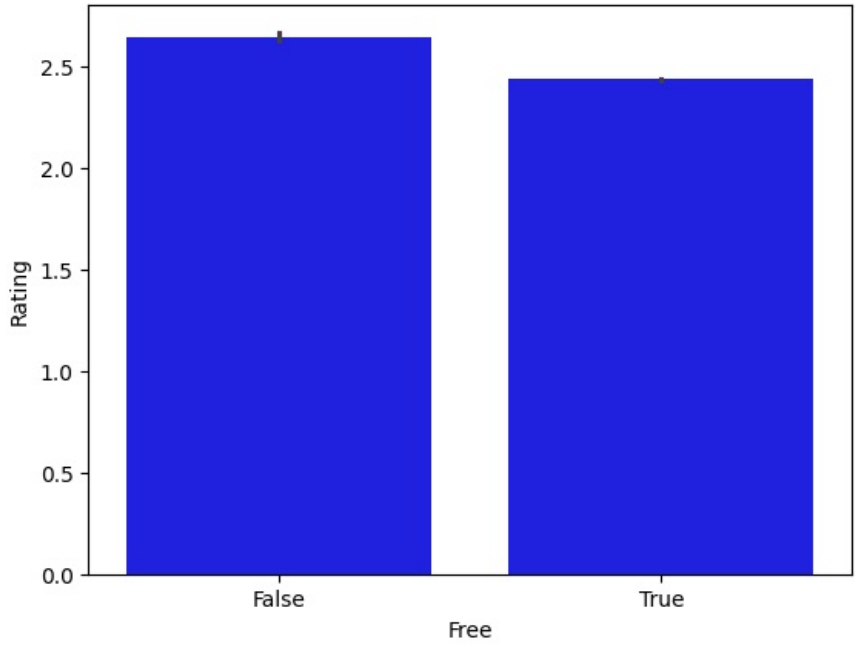


Ans 4: Tools, Communication, Productivity, Photography, Video Players & Editors have the most number of downloads.

Q5. How does the price of an app (free vs. paid) affect its rating?

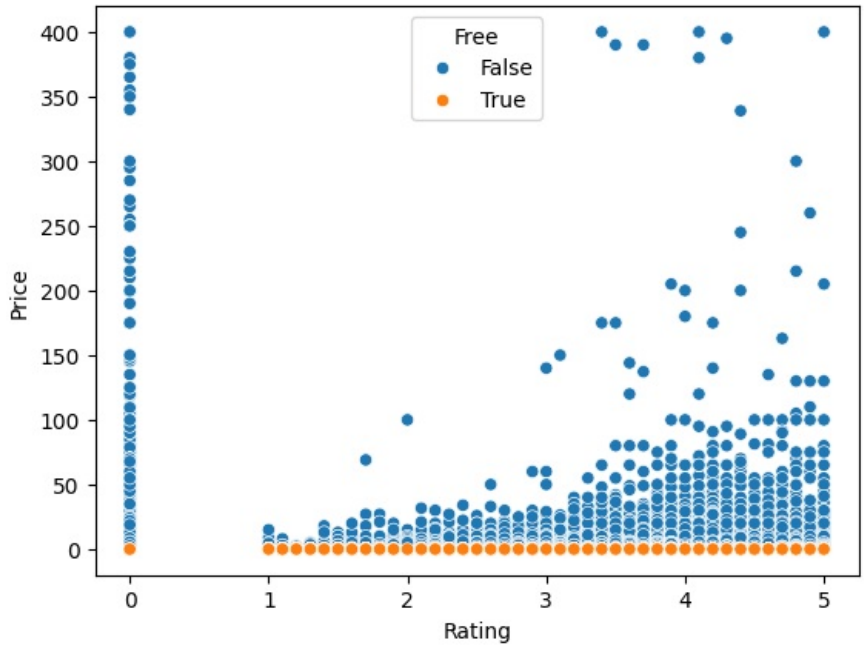
In [53]:
```
sns.barplot(data=df,x="Free",y="Rating",color="blue")
```

Out[53]:
```
<Axes: xlabel='Free', ylabel='Rating'>
```



In [54]:
```
sns.scatterplot(data=df,x="Rating",y="Price",hue="Free")
```

Out[54]:
```
<Axes: xlabel='Rating', ylabel='Price'>
```

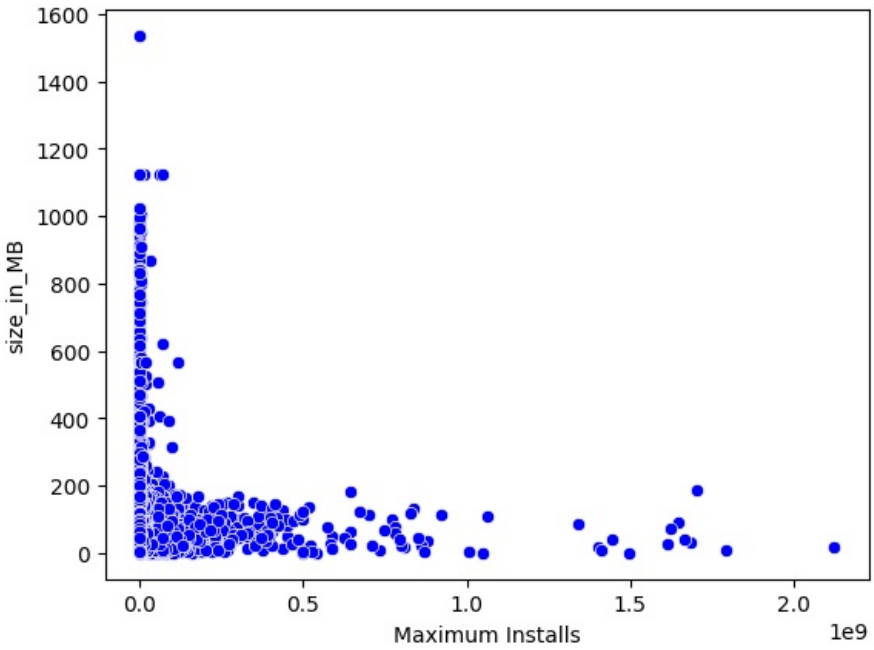


Ans 5: Rating of apps is independent of their price.

Q6. How does app size relate to installs?

In [55]:
```
sns.scatterplot(data=df,x="Maximum Installs",y="size_in_MB",color="blue")
```

Out[55]:
```
<Axes: xlabel='Maximum Installs', ylabel='size_in_MB'>
```

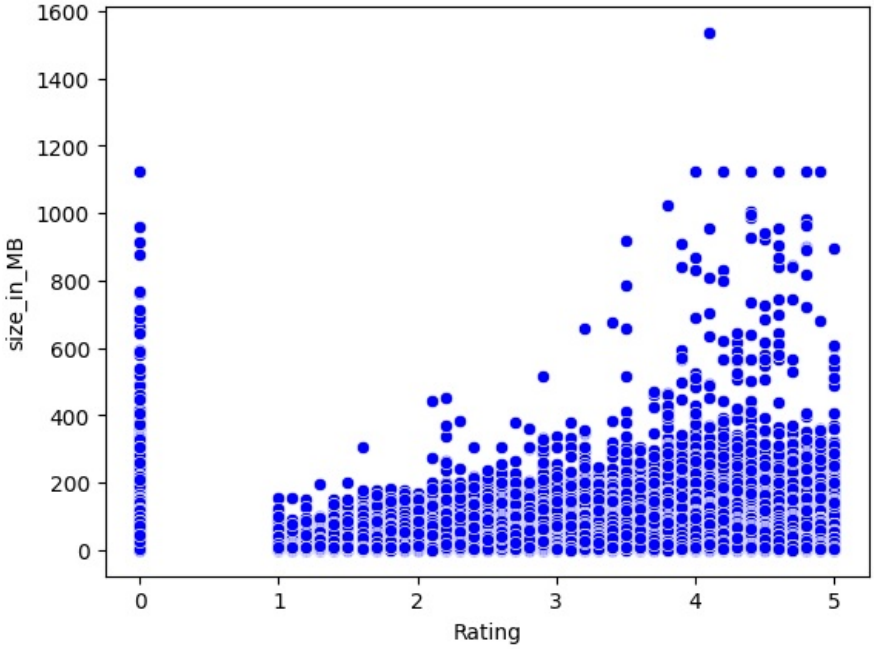


Ans 6: Apps with less than 200 MBs gets more downloads

Q7. Are Size and Rating dependent on each other?

In [56]:
```
sns.scatterplot(data=df,x="Rating",y="size_in_MB",color="blue")
```

Out[56]:
```
<Axes: xlabel='Rating', ylabel='size_in_MB'>
```



Ans 7: Apps with high size and installs seems to have descent rating.

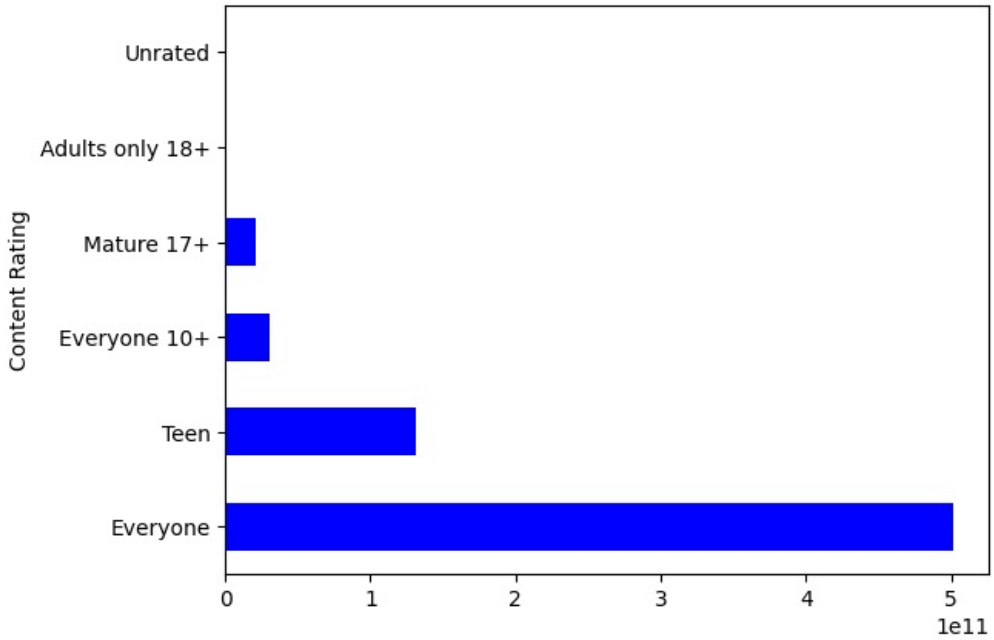
Q8. Does the content rating of an app (e.g., Everyone, Teen, Mature) influence its install base?

In [57]:
```
df.groupby("Content Rating")["Maximum Installs"].sum().sort_values(ascending=False)
```

Out[57]:
```
Content Rating
Everyone           500867597158
Teen               131529652823
Everyone 10+        30428061688
Mature 17+          20938512547
Adults only 18+        28425168
Unrated                 4081365
Name: Maximum Installs, dtype: int64
```

In [58]:
```
df.groupby("Content Rating")["Maximum Installs"].sum().sort_values(ascending=False).plot(kind="barh",color="blu
```

Out[58]:
```
<Axes: ylabel='Content Rating'>
```



Ans 8: Content Rating for everyone have highest installs after that category of teens is more powerfull in terms of insall base.

Q9. Which Year has the most number of apps released?

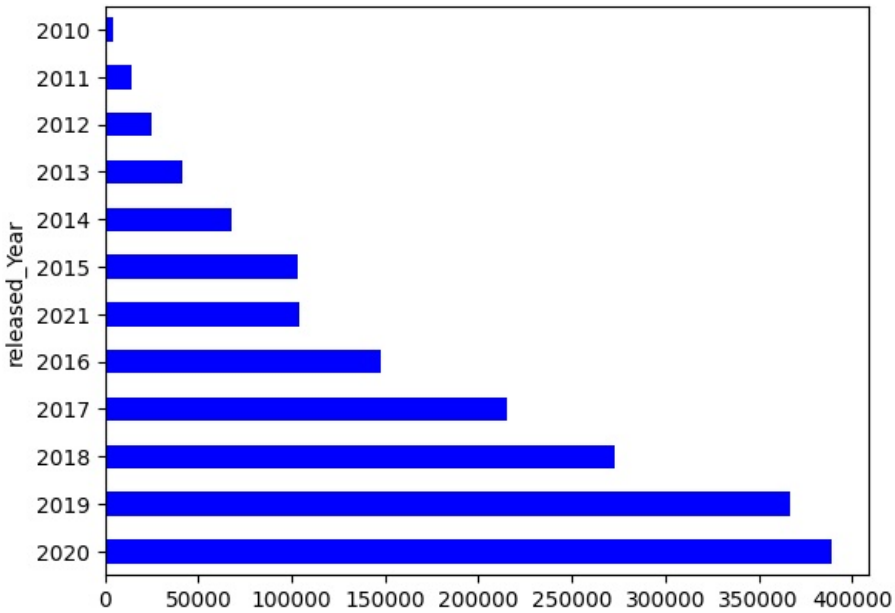
In [59]:
```
# generating new column of year from released date
df['released_Year'] = pd.to_datetime(df['Released']).dt.year.sort_values(ascending=False)
```

In [60]:
```
df["released_Year"].value_counts().sort_values(ascending=False)
```

Out[60]:
```
released_Year
2020    389158
2019    366582
2018    272648
2017    215067
2016    147594
2021    104128
2015    103568
2014     67556
2013     41612
2012     25017
2011     13990
2010      4566
Name: count, dtype: int64
```

In [61]:
```
# plot to show number of apps released, according to year
df["released_Year"].value_counts().sort_values(ascending=False).plot(kind="barh",color="blue")
```

Out[61]:
```
<Axes: ylabel='released_Year'>
```



Ans 9: Till this data most number of apps released were in 2020.

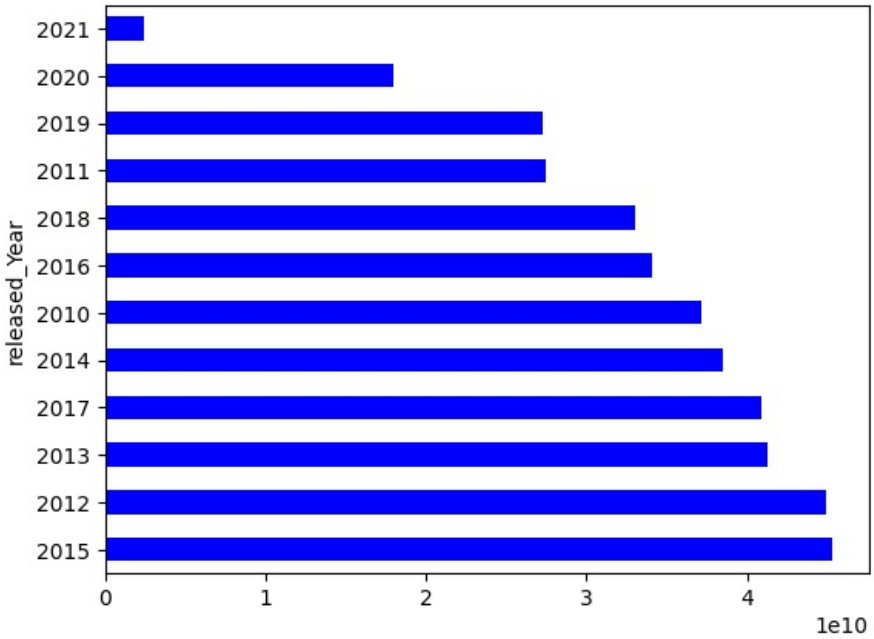
Q10. Apps released in Which Year could the most number of app downloads?

In [62]:
```
ch=df.groupby("released_Year")["Minimum Installs"].sum().sort_values(ascending=False)
ch
```

Out[62]:
```
released_Year
2015    4.533025e+10
2012    4.496979e+10
2013    4.130224e+10
2017    4.094372e+10
2014    3.855627e+10
2010    3.712798e+10
2016    3.408687e+10
2018    3.302314e+10
2011    2.752461e+10
2019    2.728610e+10
2020    1.800780e+10
2021    2.466482e+09
Name: Minimum Installs, dtype: float64
```

In [63]:
```
ch.plot(kind="barh",color="blue")
```

Out[63]:
```
<Axes: ylabel='released_Year'>
```

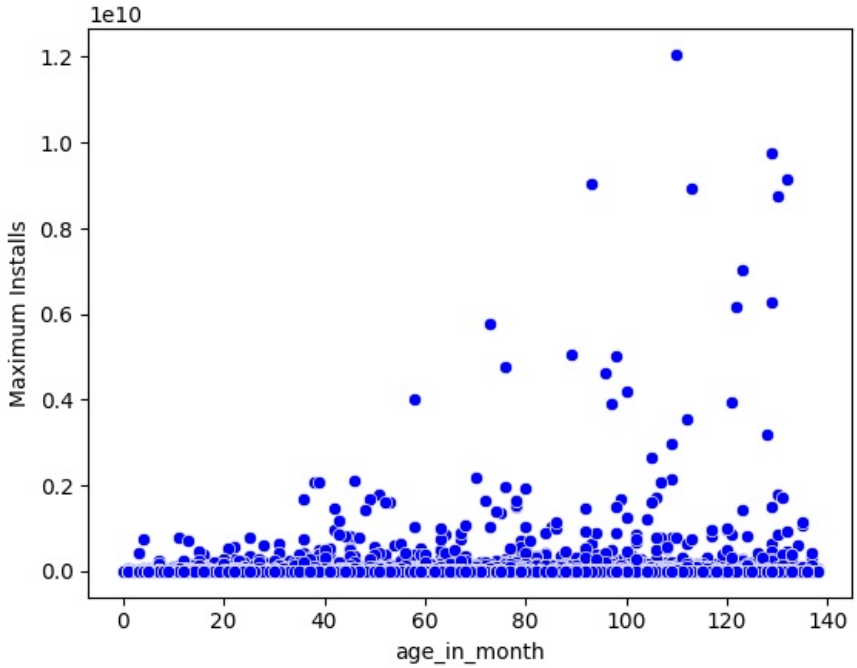


Ans 10: Most downloads from playstore has been achieved by apps released in 2015.

Q11: Does app age influences it's downoads?

In [64]:
```
sns.scatterplot(df,x="age_in_month",y="Maximum Installs",color="blue")
```

Out[64]:
```
<Axes: xlabel='age_in_month', ylabel='Maximum Installs'>
```



In [65]:
```
df[["age_in_month","Maximum Installs"]].corr()
```

Out[65]:

	age_in_month	Maximum Installs
age_in_month	1.00000	0.02302
Maximum Installs	0.02302	1.00000

Ans 11: No, app age donot inflence it's downloads.

Q12. Show interdependency of all numerical features

In [66]:
```
sns.heatmap(df[["Rating","Rating Count","Minimum Installs","Maximum Installs","Price","size_in_MB","released_Yea
```

Out[66]: <Axes: >

Ans 12: As the value of "Maximum Installs" increases, the value of the "Rating Count" tends to increase as well, but not always by the same amount.

7.0 Summary

The EDA exercise conducted on the Google Play Store dataset has yielded numerous interesting insights. The dataset was found to be relatively clean and consistent throughout the analysis. We posed several questions to the dataset and provided detailed answers and findings as follows:

Q1. Is there any correlations between rating and dowloads?

Ans 1: Not seen any relationship in apps with less downloads, but it is seen that apps with more downloads have quite more rating.

Q2: Is there any relationship between downloads and app type(free/paid)?

Ans 2: Free apps generally have more downloads than that of paid ones

Q3. Which TOP 5 app categories have the most number of apps?

Ans 3: Education, Music & Audio, Tools, Business and Entertainment have the most number of apps.

Q4. Which TOP 5 app categories have the most downloads?

Ans 4: Tools, Communication, Productivity, Photography, Video Players & Editors have the most number of downloads.

Q5. How does the price of an app (free vs. paid) affect its rating?

Ans 5: Rating of apps is independent of their price.

Q6. How does app size relate to installs?

Ans 6: Apps with less than 200 MBs gets more downloads.

Q7. Are Size and Rating dependent on each other?

Ans 7: Apps with high size and installs seems to have descent rating.

Q8. Does the content rating of an app (e.g., Everyone, Teen, Mature) influence its install base?

Ans 8: Content Rating for everyone have highest installs after that category of teens is more powerfull in terms of insall base.

Q9. Which Year has the most number of apps released?

Ans 9: Till this data most number of apps released were in 2020.

Q10. Apps released in Which Year could get the most number of app downloads?

Ans 10: Most downloads from playstore has been achieved by apps released in 2015.

Q11: Does app age influences it's downoads?

Ans 11: No, app age donot inflence it's downloads.

Q12. Show interdependency of all numerical features.

Ans 12: As the value of "Maximum Installs" increases, the value of the "Rating Count" tends to increase as well, but not always by the same amount.

08 Contact Details

Click on link below to contact/follow/correct me:

- **Email:** waqasliaqat630@gmailcom
- Linkedin
- Github
- kaggle

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js